

VLA 강의 — 발표 대본

RT2에서 π_0 까지, 그리고 그 너머

Giseop Kim

APRL, DGIST

출처 (Source)

본 강의 자료는 다음 영상을 기반으로 재구성되었습니다.

- **제목:** Inside the World's Smartest Robot Brain [VLA]
- **저자:** [Welch Labs](#)
- **영상 URL:** youtube.com/watch?v=2mrGMMmrVNE

이 문서에 대하여

- 이 문서는 vla_lecture.tex(슬라이드 본편)와 **1:1로 대응**되는 발표 대본입니다.
- 한 슬라이드당 **2-4분 분량**의 구어체 설명을 담았습니다 (총 약 60-90분 강의 분량).
- 강의 시 vla_lecture.pdf를 띄워놓고 이 문서를 **진행 노트**로 활용하시면 됩니다.
- 대본은 그대로 읽기보다는 **흐름과 강조점을 잡는 가이드**로 사용하기를 권장합니다.

표기

- 각 슬라이드 대본 시작 부분에 [**슬라이드 번호 / 제목**] 표시
- 기울임체: 청중의 반응을 유도하는 질문 또는 강조 톤

말줄임 : 호흡을 두는 자리

Part 1. Coke Can과 Taylor Swift

[3] 2023년 7월: 어색했지만 결정적인 데모

자, 본격적으로 들어가 봅시다. 시계를 2023년 7월로 돌려볼게요.

Google의 한 연구실, 평범한 책상 위에 코크 캔 하나, 그리고 Tom Cruise, Snoop Dogg, Taylor Swift의 사진 세 장이 놓여 있습니다. 연구자가 로봇에게 명령을 내립니다. “Move the Coke can to Taylor Swift.”

이 명령을 받은 로봇은 RT2라는 새로운 두뇌로 움직였는데요, 흥미로운 점은 RT2가 너무 커서 **로봇 안에 올릴 수가 없었다**는 겁니다. 그래서 카메라 영상을 한 장씩 클라우드의 TPU 클러스터로 보내고, 거기서 제어 신호를 받아오는 방식이었어요.

그렇게 천천히, 정말 어색하게, 로봇이 코크 캔을 들어서 Taylor Swift 사진의 가장자리에 슬쩍 내려놓았습니다.

[말줄임] 솔직히 말하면 영상을 보면 시시해 보입니다. 그런데 2년 후, 당시 팀 멤버였던 Carol Houseman이 인터뷰에서 이 장면을 이렇게 회고합니다. “이게 정말 될 거라는 확신이 든 순간이었어요.”

그 후 1년 안에 RT2 핵심 멤버 대부분이 Google을 떠나서 **Physical Intelligence**라는 스타트업을 차리게 됩니다. 이 어색한 데모가 어떻게 한 회사를 탄생시킬 만한 사건이었을까요? 다음 슬라이드에서 그 답을 봅시다.

[4] 왜 이 어색한 데모가 돌파구였나

핵심은 한 가지 사실에 있습니다. **로봇 학습 데이터에 Taylor Swift는 단 한 번도 등장하지 않았다**는 것.

생각해 보세요. RT2가 이 명령을 수행하려면 두 종류의 지식을 동시에 갖고 있어야 합니다.

첫째, “Taylor Swift는 사람이고, 이 얼굴이 Taylor Swift다”라는 의미적 지식. 이건 인터넷 사전학습에서 옵니다. 위키피디아, 뉴스 기사, 이미지에 달린 캡션, 이런 데이터요.

둘째, “코크 캔을 집어서 어떤 위치로 옮긴다”는 물리적 행동. 이건 인간이 로봇을 조작한 시연 데이터에서 옵니다.

이 둘은 **전혀 다른 종류의 데이터**예요. 한쪽은 텍스트와 이미지, 다른 쪽은 관절 각도와 그리퍼 명령. RT2가 이 작업을 해냈다는 건, 모델 안에서 이 두 세계가 **연결**되었다는 뜻입니다.

[말줄임] 이게 왜 중요한가? 만약 정말로 이게 가능하다면, 우리가 가진 **인터넷 전체의 지식**을 로봇 두뇌에 부어 넣을 수 있다는 얘기거든요. 학습 데이터를 일일이 수집할 필요가 없어진다는 의미죠. 위키피디아에 있는 모든 개념이 로봇이 활용할 수 있는 지식이 되는 겁니다.

대학원생 입장에서 생각해 봅시다. 여러분이 로봇 학습 데이터셋을 만든다고 하면 가장 큰 병목이 뭐죠? 데이터 수집입니다.

VLA는 그 병목을 우회할 가능성을 보여준 첫 번째 정량적 증거였어요.

Part 2. Google의 진화 경로

[5] 2022: SayCan — LLM을 플래너로

이제 RT2가 어떤 흐름에서 나왔는지를 봅시다. 출발점은 2022년의 SayCan이에요.

ChatGPT가 막 나오던 시기죠. Google 연구자들이 자연스럽게 떠올린 질문이 있었어요. “LLM이 이렇게 똑똑한데, 로봇 플래닝에 쓸 수 없을까?”

SayCan의 아이디어는 간단합니다. 사람이 “얹질러진 것을 닦아라”라고 말하면, LLM이 이걸 단계별로 쪼개주는 거예요. “스펀지를 찾아라, 스펀지를 집어라, 얹질러진 곳으로 가라, 닦아라.” 이렇게요.

근데 여기 결정적인 한계가 있었습니다. **실제 제어는 별도의 imitation learning 네트워크가 했다**는 점이에요. LLM은 “스펀지를 집어라”라고 말만 할 수 있고, 실제로 그게 가능하려면 별도의 네트워크가 “스펀지 집기”를 미리 학습해 뒀야 했습니다.

즉, LLM은 **미리 정해진 메뉴판**에서 항목을 고르는 역할일 뿐이었어요. 메뉴에 없는 행동은 못 합니다. 코크 캔을 Taylor Swift 사진에 올리려면? 그 행동을 미리 학습시켜야 했어요.

[말줄임] 또 하나 중요한 한계는, SayCan에서 **LLM이 눈을 감고 있었다**는 거예요. 텍스트 전용 모델이라 카메라 이미지를 볼 수 없었거든요. 이게 나중에 PaLM-E로 풀리는 문제입니다.

[6] 2022 말: RT1 — 더 큰 행동 메뉴

SayCan의 메뉴 문제를 해결하려면 어떻게 해야 할까요? 답은 단순합니다. **메뉴를 키운다.**

2022년 말 Google이 발표한 RT1, Robot Transformer 1이 정확히 그 일을 했어요. 같은 imitation learning 패러다임이지만, 데이터를 **13만 건 이상**으로 대규모로 모았습니다. 그리고 transformer 기반 아키텍처를 도입했죠.

이게 SayCan의 플래너 LLM에게 훨씬 풍부한 “할 줄 아는 것들의 목록”을 제공해 주었습니다. 처음 보는 주방에서 특정 물건을 찾는 작업처럼, 여러 단계가 필요한 long-horizon task에서 성능이 눈에 띄게 좋아졌어요.

[말줄임] 그런데도 한 가지 문제가 여전히 남아 있었어요. 플래너 LLM은 **여전히 텍스트만 봅니다**. 시각 정보가 없어요. “서랍을 열어라”라고 계획을 세웠는데 누가 이미 서랍을 열어놨다? LLM은 모릅니다. 계속 “서랍을 열어라”라고 지시할 거예요.

이게 다음 단계인 PaLM-E의 동기가 됩니다.

[7] 2023년 3월: PaLM-E — 시각을 가진 플래너

2023년 3월 6일. GPT-4 출시 약 일주일 전이에요. Google이 PaLM-E를 발표합니다.

PaLM-E는 한 줄로 요약하면, **PaLM에 이미지 입력을 직접 통합한 멀티모달 LLM**이에요. 이제 플래너가 카메라를 통해 세상을 볼 수 있게 된 겁니다.

이게 얼마나 큰 차이를 만들었는지, 데모 하나를 소개할게요. 로봇에게 “칩 봉지를 가져와라”라고 지시합니다. 로봇이 서랍을 열고 칩 봉지를 꺼내려는 순간, 연구자가 장난스럽게 봉지를 다시 서랍에 집어넣어요.

기존 시스템이라면 로봇은 자기가 이미 “꺼냈다”고 생각하고 다음 단계로 넘어가겠죠. 그런데 PaLM-E는 **상황이 바뀌었음을 인식하고 계획을 다시 세웁니다**. 다시 서랍을 열고, 다시 봉지를 잡고, 그것도 자율적으로요.

[말줄임] 이게 **visual feedback이 closed loop으로 들어온** 첫 번째 사례에 가깝습니다. 플래너가 매 순간 “지금 세상이 어떤가?”를 보면서 계획을 적응시키는 거죠.

스택은 여전히 두 단계예요. PaLM-E가 플래닝하고, RT1이 제어합니다. 하지만 두 단계 모두 비전이 들어왔다는 게 큰 변화입니다.

[8] 관찰: 두 모델이 너무 닮았다

여기서 흥미로운 관찰이 시작됩니다. 잠깐, PaLM-E랑 RT1이 너무 비슷하지 않나?

두 모델 다 비전 인코더가 있고, transformer가 있고, 입력으로 이미지와 언어를 받아요. 차이는 단 하나, 출력입니다. PaLM-E는 텍스트를 출력하고, RT1은 제어 신호를 출력합니다.

자, 그럼 자연스러운 질문이 나오죠. “하나의 모델이 둘 다 출력하면 안 될까?”

생각해 보세요. transformer의 출력은 결국 토큰 시퀀스예요. 텍스트 토큰을 출력하는 모델에, 행동을 표현하는 **action token**을 vocabulary에 추가하면 어떨까요? “책상으로 가라”고 텍스트로 출력할 수도 있고, 관절 각도를 정수로 인코딩한 action token을 출력할 수도 있는, 그런 통합 모델을 만든다면?

[말줄임] 이게 단순히 엔지니어링 트릭이 아닙니다. 더 근본적인 질문이에요. 지금까지 우리가 만든 가장 강력한 AI인 LLM이, 그 자체로 로봇 두뇌가 될 수 있는가?

이 질문에 답을 시도한 게 다음 단계, RT2입니다.

[9] 2023년 7월: RT2 — 통합의 탄생

2023년 7월. Google 로보틱스 팀이 RT2를 공개합니다. 레시피는 의외로 직설적이에요.

1단계: 멀티모달 LLM에서 출발한다. 구체적으로는 PaLM-E와 PaLI-X 두 가지를 베이스로 썼어요.

2단계: 6개월 전 RT1 학습에 사용했던 똑같은 인간 시연 데이터를 가져온다.

3단계: 단, 출력을 토큰화된 제어 신호로 변환해서 fine-tune한다. 즉, “오른팔을 5도 회전하라”를 하나의 토큰으로 표현하고, LLM이 이걸 텍스트 토큰처럼 출력하도록 학습시킨 거죠.

이게 됩니다. 그것도 **학습 데이터에 없던 객체, 환경, 작업까지 일반화**하면서요. 우리가 처음 본 Taylor Swift 데모가 바로 그 결과입니다.

[말줄임] 팀은 이 새로운 종류의 모델에 이름을 붙입니다. **Vision-Language-Action**, 줄여서 VLA. 비전, 언어, 행동이 단일 모델로 통합됐다는 뜻이에요.

한 가지 강조할 게 있습니다. RT2 모델군은 50억에서 550억 파라미터 규모였어요. **로봇에 직접 올릴 수 없는 크기**입니다. 그래서 영상에서도 클라우드 추론을 했죠. 이게 다음 단계인 π_0 의 또 다른 출발점이 됩니다.

Part 3. π_0 아키텍처 해부

[10] 2024년 10월: π_0 — 더 작고 더 빠르고 더 잘하는

RT2 데모로부터 15개월이 지난 2024년 10월. Physical Intelligence 팀이 π_0 , “Pi Zero”를 공개합니다.

차이가 인상적이에요. RT2는 코크 캔을 사진 위에 올리는 정도였다면, π_0 는 **빨래를 건조기에서 꺼내 개고, 식탁을 정리하고, 그릴드 치즈 샌드위치를 만들고, 처음 보는 침실까지 정리**합니다. 정말로 “집안일을 도와주는 로봇”에 가까워졌어요.

자, 그럼 π_0 가 RT2보다 **더 큰 모델**일까요? 직관적으로는 그렇게 생각하기 쉽죠.

[말줄임] 정답은 반대입니다. RT2가 50억~550억 파라미터였는데, π_0 는 **33억 파라미터**예요. 더 작아졌습니다. 그리고 클라우드 TPU가 아니라 **컨슈머급 RTX 4090** 한 장으로 **73 ms**만에 추론합니다. 로봇 위에 직접 올라간다는 거죠.

어떻게 더 작은 모델이 더 잘할 수 있을까요? 답은 “아키텍처가 더 영리해졌다”에 있습니다. 다음 슬라이드에서 그 영리함을 봅시다.

[11] π_0 전체 구조

π_0 아키텍처를 한눈에 봅시다. 입력은 네 종류예요.

왼쪽 상단: 카메라 세 대(머리 위 + 양 손목)와 텍스트 프롬프트. 이건 **PaliGemma**로 들어갑니다. PaliGemma는 Google이 공개한 오픈 가중치 멀티모달 LLM이에요. SigLIP 이미지 인코더와 Gemma LLM이 결합된 형태죠.

왼쪽 하단: 로봇의 현재 상태(관절 각도 14개)와 노이즈 섞인 미래 행동들. 이건 **Action Expert**로 들어갑니다.

오른쪽: Action Expert가 14개 관절 $\times$ 50 시간 단계의 궤적을 출력합니다.

여기서 **핵심은 두 모듈을 잇는 점선 화살표**입니다. “KV cache”라고 적혀 있는 부분이요. PaliGemma가 계산한 정보가 Action Expert로 직접 전달되는 통로입니다. 이걸 어떻게 하는지가 π_0 의 영리함의 중심이에요.

[말줄임] 슬라이드 아래의 세 가지 설계 결정을 강조하고 싶어요. 첫째, LLM 백본과 Action Expert를 분리. 둘째, flow matching으로 행동 생성. 셋째, KV cache로 두 모듈 결합. 다음 슬라이드들에서 하나씩 풀어봅시다.

[12] 설계 결정 1: 왜 Action Expert를 따로 두는가

첫 번째 설계 결정. 왜 LLM이 직접 action token을 출력하지 않고, 별도의 Action Expert를 두는가?

비유로 시작해 봅시다. 인간 뇌에서 “전두엽의 인지”와 “운동 피질의 운동 제어”는 다른 영역이 담당하잖아요. 분리되어 있지만 긴밀하게 협업합니다. π_0 도 비슷한 철학이에요. **Gemma는 “인지”에, Action Expert는 “운동”에 특화시킨** 거죠.

흥미로운 디테일 하나 말씀드릴게요. Action Expert의 정체가 뭐냐 하면—**Gemma와 똑같은 아키텍처**예요. 실제 코드에서도 Gemma 클래스로 인스턴스화됩니다. 차이는 두 가지뿐이에요. **무작위로 초기화된다는 점, 그리고 각 layer의 width가 더 작다**는 점입니다. 사전학습이 없으니 처음부터 행동에 맞게 학습되고, 작아서 가벼워서 실시간에 유리하죠.

[말줄임] 여기서 “어, 이거 SayCan으로 돌아간 거 아니야?” 하실 수 있어요. 둘 다 상위 모듈이 “계획”하고 하위 모듈이 “제어”하니깐요. **핵심적인 차이는 인터페이스에 있습니다.**

SayCan에서는 두 모듈이 **자연어로 통신**했어요. “스펀지를 집어라”라는 텍스트로요. 이건 **매우 좁은 통로**입니다. 정보 손실이 커요.

π_0 에서는 두 모듈이 **attention 내부 표현으로 통신**합니다. 고차원 벡터 전체가 그대로 흐르는 거예요. 이게 “하나의 두뇌처럼 사고하면서도 모듈성을 유지한다”는 말의 의미입니다.

Part 4. 내부 동작: Attention head

[13] Gemma LLM이 받는 입력

이제 모델 내부로 들어갑시다. Gemma LLM이 무엇을 입력으로 받는지부터요.

이미지 처리부터 봅시다. 카메라가 세 대 있어요—머리 위 1대, 양쪽 손목 1대씩. 각 이미지는 패치 그리드로 쪼개져서 이미지당 256개 패치가 됩니다. 세 장이니까 총 **768 패치**예요.

각 패치는 이미지 인코더(SigLIP)를 거쳐서 길이 2048짜리 임베딩 벡터로 변환됩니다. 이것 **soft token**이라고도 불러요. “부드럽게 인코딩된 토큰” 정도의 의미죠.

텍스트 처리는 익숙하실 거예요. “uncap the pen” 같은 프롬프트가 4개 토큰으로 쪼개지고, 각 토큰이 임베딩 벡터로 매핑됩니다.

다 합치면 $768 + 4 = 772$ 개의 **soft token**이 Gemma로 들어가는 거예요.

[말줄임] Gemma 자체는 표준 transformer입니다. 18개 블록이 쌓여 있고, 각 블록은 attention과 MLP로 구성돼요. 각 attention 블록 안에는 8개의 head가 있습니다. 다음 두 슬라이드에서 이 head 하나가 무엇을 하는지 자세히 들여다볼 거예요.

[14] Attention head 한 개의 연산

Attention 한 head가 하는 연산을 수식으로 정리해 봅시다. 익숙한 분도 많겠지만, 다음 슬라이드의 “pen” 사례를 이해하기 위해 짚고 갑니다.

입력: 772개의 임베딩 벡터를 행으로 쌓은 행렬 X . 이게 출발점이에요.

1단계 — 세 행렬 생성. X 에 세 개의 학습 가능한 가중치 행렬을 곱해서 query Q , key K , value V 세 행렬을 만듭니다. 각각 772행이에요. 입력 토큰 하나당 query 한 개, key 한 개, value 한 개씩이죠.

2단계 — Attention pattern. Q 와 K^T 를 곱하고 $\sqrt{d_k}$ 로 나눈 뒤 softmax를 씁니다. 결과는 772×772 행렬이에요. i 번째 행, j 번째 열의 값은 “ i 번째 토큰이 j 번째 토큰에 얼마나 주목하는가”를 나타냅니다.

3단계 — 출력. attention pattern과 V 를 곱합니다. 직관적으로는 각 query가 “나에게 관련 있는 정보가 어디 있나?”를 묻고, 그 위치의 value를 자기 자리에 가져오는 연산이에요.

[말줄임] 한 가지 더 강조할게요. Q 와 K 의 dot product가 큰 곳일수록 정보가 많이 “복사”되는 겁니다. 다음 슬라이드에서 이 직관이 어떻게 “펜 찾기”로 구현되는지 봅시다.

[15] 사례 연구: “pen” 토큰이 펜을 찾는다

이제 진짜 흥미로운 부분이에요. 학습된 attention head 하나를 **현미경으로 들여다본 결과**입니다.

시나리오: 프롬프트가 “uncap the pen”이고, 마지막 토큰 “pen”에 대응하는 query 벡터를 q_{pen} 이라고 합시다.

이 head에서 무슨 일이 일어나느냐. q_{pen} 과 모든 768개 image patch의 key 벡터 사이 dot product를 계산합니다. 결과를 보면 **거의 모든 패치는 작은 값**이고, **딱 두 패치에서만 값이 압도적으로 큼**니다. 그 두 패치가 어디냐? **실제로 펜이 보이는 그 두 패치**예요.

Softmax를 거쳐서 attention 값을 이미지 위에 히트맵으로 입혀 보면—펜이 있는 자리만 빨강게 빛납니다. 다른 곳은 거의 검은색이고요.

여기서 **진짜 놀라운 점**이 나와요. 이 분석을 비디오의 모든 프레임에 대해 실행합니다. 그러면 어떻게 되겠어요? **펜이 자동으로 추적됩니다**. 별도의 추적 알고리즘 없어요. 그냥 attention head 하나가 그 일을 하는 거예요.

[말줄임] 이게 의미하는 바가 뭐냐. 이 모델은 **명시적으로 “펜을 찾아라”라고 학습된 적이 없어요**. 그냥 인간 시연 데이터로 행동을 학습했을 뿐인데, 내부적으로 “프롬프트의 명사를 이미지에서 grounding하는 기능”이 **창발**한 겁니다.

마지막으로, 이건 단 하나의 head 이야기예요. $18 \text{ layer} \times 8 \text{ head} = 144$ 개의 head가 각자 다른 패턴을 학습하고 있습니다.

[16] Attention head의 의미론적 해석

앞 슬라이드에서 “attention 값이 크면 정보가 복사된다”고 말씀드렸어요. 이걸 정확히 짚고 넘어갑시다.

수식으로 보면 출력 위치 i 의 값에 a_{ij} 곱하기 v_j 가 더해집니다. 즉, “pen” 토큰 위치 i 에 **펜 patch**의 value 벡터 v_j 가 **복사되어 합쳐지는** 거예요.

이걸 의미적으로 해석하면—**이 attention head는 텍스트의 “pen”이라는 단어와, 이미지 속 펜이라는 시각적 객체를 하나의 통합 표현으로 묶어내고 있는** 겁니다. 언어와 시각의 grounding이 모델 내부 어딘가에서 이런 식으로 일어나고 있다는 거예요.

[말줄임] 그리고 다시 강조하지만, 우리가 본 건 **144개 중 단 하나의 head**예요. 다른 head들은요? 어떤 head는 “손과 물체의 거리”에 주목할 수 있고, 어떤 head는 “현재 작업의 진행 단계”를 추적할 수 있고, 어떤 head는 “시간적 일관성”을 담당할 수 있어요.

연구자들이 이런 “mechanistic interpretability” 작업을 통해 LLM 내부에서 무슨 일이 벌어지는지를 조금씩 이해해 가고 있는데, VLA 모델에서도 이런 분석이 점점 더 중요해질 겁니다. 우리 lab에서 이런 방향으로 연구해 보고 싶은 분 있으면 따로 얘기해 봐요.

Part 5. Action Expert와 Flow Matching

[17] Action Expert의 입력

이제 Action Expert로 넘어갑니다. 입력이 두 종류예요.

입력 1: 로봇 상태 (1 token). Aloha 플랫폼에서 각 팔은 7개 관절(허리, 어깨, 팔꿈치, 전완 회전, 손목, 손목 회전, 그리퍼)을 가져요. 양 팔 합치면 **14차원 벡터**입니다. 이 14차원 벡터를 14×1024 가중치 행렬로 곱해서, 길이 **1024**짜리 임베딩 벡터 하나로 변환합니다.

여기서 짚고 넘어갈 점. Gemma의 임베딩은 길이 2048인데, Action Expert는 1024예요. **차원이 절반**이죠. 왜? **연산량과 추론 시간을 줄이기 위해서**입니다. 작은 모델로 충분하다는 판단인 거예요.

입력 2: 노이즈가 섞인 미래 행동 (50 tokens). 다음 50 시간 단계의 14차원 관절 위치 예측이 50개 토큰으로 들어갑니다. 처음에는 **순수한 노이즈**예요.

[말줄임] 잠깐, “예측할 행동을 입력으로 받는다”는 게 직관적으로 이상하시죠? 이게 다음 슬라이드의 핵심 주제, flow matching입니다. 미리 예고하면, 이건 **이미지 생성 모델**에서 가져온 아이디어예요.

[18] Flow Matching: 노이즈에서 궤적으로

Flow matching의 직관은 단순합니다. 이미지 생성에서 “노이즈에서 시작해 점점 다듬으면 고양이 사진이 나온다”는 그 메커니즘 있잖아요. π_0 는 같은 메커니즘으로 “노이즈에서 시작해 점점 다듬으면 14×50 관절 궤적이 나온다”를 합니다.

왜 이 방법이 잘 작동하느냐? 핵심은 분포가 multimodal이라는 사실에 있어요.

이미지 생성에서 “고양이 사진”이라는 분포를 생각해 보세요. 한 가지 “올바른 고양이 이미지”가 아니라, 무수히 많은 가능한 고양이 이미지가 존재합니다. 정면 고양이, 옆모습 고양이, 자는 고양이, 점프하는 고양이...

로봇 행동도 똑같습니다. “펜 뚜껑을 여는 방법”은 한 가지가 아니에요. 어느 손으로? 어느 각도로? 어느 속도로? 무수히 많은 valid trajectories가 있습니다.

[말줄임] 만약 단순한 regression으로 학습하면, 모델은 모든 valid trajectories의 평균을 출력하는 경향이 있어요. 그게 어떻게 보일까요? 어색하고 비현실적인 동작이 됩니다. 두 valid 동작의 평균이 valid한 동작이 아닌 거죠. 이것 “mode collapse”라고 합니다.

Flow matching은 이 문제를 우회합니다. 평균을 학습하지 않고, 분포 자체를 학습하니까요. 매번 여러 valid mode 중 하나를 자연스럽게 샘플링합니다.

π_0 의 절차를 정리하면—무작위 14×50 노이즈에서 출발해서, Action Expert가 “이걸 어떻게 업데이트해야 더 그럴듯한 궤적이 되는지”를 예측하고, 더해주고, 다시 모델에 넣고, 또 업데이트하고. 이것 10번 반복하면 매끄러운 궤적이 나옵니다.

[19] 이미지 생성과 로봇 제어의 통합

잠시 한 발짝 떨어져서 봅시다. 우리가 방금 본 사실이 사실 굉장히 깊은 통찰을 담고 있어요.

이미지 생성에서: 텍스트와 노이즈 이미지를 입력으로 받아서, 자연 이미지의 manifold 위의 한 점을 출력합니다. “cat”이라는 텍스트가 cat 이미지를 가리키도록요.

로봇 제어 (π_0)에서: 프롬프트, 이미지, 노이즈 궤적을 입력으로 받아서, 가능한 동작 manifold 위의 한 점을 출력합니다. “uncap the pen”이라는 텍스트가 펜 따는 궤적을 가리키도록요.

같은 수학, 다른 응용이에요. 같은 flow matching 손실 함수, 같은 디노이징 절차, 같은 transformer 백본. 다른 건 데이터의 종류뿐입니다.

[말줄임] 이게 우연일까요? 아니라고 봅니다. **매우 다른 응용 영역에서 같은 수학적 프레임워크가 작동한다**는 건, 그 프레임워크가 어떤 **심오한 통일성**을 포착하고 있다는 신호예요.

실제로 이 통찰이 후속 연구의 토대가 되었습니다. Diffusion Policy, RDT, Octo 같은 모델들이 모두 이 “생성 모델 = 정책 학습” 패러다임을 채택하고 있어요. 우리 lab의 ScaleMaster나 LT-Mem 같은 작업에서도 이 흐름을 어떻게 활용할지 고민해 볼 수 있습니다.

Part 6. 두 모듈의 결합 — KV cache 트릭

[20] 문제 설정

이제 가장 영리한 부분으로 갑시다. 두 모듈을 어떻게 연결하느냐.

상황을 정리해 봅시다. Action Expert도 Gemma와 같은 구조예요. $18 \text{ layer} \times 8 \text{ head}$ 를 가집니다.

Action Expert 입장에서, 자기 query는 **무엇을 “보고” 싶을까요?** 네 가지가 있어요.

첫째, 다른 시간 단계의 행동들. 부드러운 궤적을 만들려면 “이전 시간 단계에서 손이 어디 있었나”를 봐야 하니까요.

둘째, 프롬프트. “무엇을 해야 하는가”를 알아야 합니다.

셋째, 이미지. “펜이 어디 있는가”를 봐야죠.

넷째, 로봇 상태. “지금 내 손이 어디 있는가”도 알아야 합니다.

[말줄임] 그런데 첫째와 넷째는 Action Expert 자기 안에 있어요. 그건 쉽습니다. 문제는 둘째와 셋째예요. 프롬프트와 이미지 정보는 **PaliGemma의 attention 안에 들어 있습니다**. 어떻게 거기에 접근하게 할까요?

이게 다음 슬라이드의 핵심 트릭, “Key/Value 이어 붙이기”입니다.

[21] 해법: Key/Value를 이어 붙이기

해법은 단순하지만 영리합니다. 한 줄로 말하면, **같은 layer index에서 두 모델의 K, V를 그냥 concatenate**한다는 거예요.

수식으로 봅시다. Action Expert의 layer ℓ 에서 attention을 계산할 때, key 행렬을 이렇게 만듭니다. Action Expert 자기 자신의 K_{ℓ}^{AE} 위에 PaliGemma의 K_{ℓ}^{Gemma} 을 쌓아서 K_{total} 을 구성하는 거예요. Value 행렬도 마찬가지로요.

크기를 보면 $51 + 772 = 823$ keys입니다. Action Expert의 query 51개가, 자기 51 토큰과 Gemma의 772 토큰을 **한 번의 attention 연산으로 동시에** 참조할 수 있게 되는 거죠.

왜 이게 가능할까요? 두 모델이 **같은 width, 같은 layer 수, 같은 head 차원**을 갖기 때문입니다. Key/value 공간의 차원이 호환되니까 그냥 이어 붙일 수 있는 거예요.

[말줄임] 이게 바로 “같은 아키텍처를 쓴다”는 설계 결정의 진짜 이유예요. 단순히 코드 재사용이 아니라, **이 KV concatenation을 가능하게 하기 위한 의도적 선택**이었던 겁니다.

다른 패러다임과 비교해 보세요. 만약 두 모델이 다른 아키텍처라면? Cross-attention layer를 추가하거나, projection layer를 뒤서 차원을 맞추거나, 별도의 연결 메커니즘을 설계해야 합니다. 여기서는 그게 다 공짜예요.

[22] 효율성: KV cache 재사용

KV concatenation의 또 다른 보너스가 있어요. 효율성입니다.

표준 LLM 추론에서 KV cache는 익숙한 개념이죠. 새 토큰이 들어올 때마다 이전 K, V를 재계산하지 않고 캐시에서 꺼내 쓰는 최적화예요.

π_0 가 이걸 영리하게 활용합니다. 절차를 보면—

1단계: PaliGemma forward pass를 한 번 돌립니다. 이 과정에서 모든 layer의 K_ℓ^{Gemma} 과 V_ℓ^{Gemma} 을 캐시해 둡니다.

2단계: Flow matching은 **10번 반복**돼야 한다고 했죠? 매 반복마다 Action Expert가 새로 forward pass를 돌립니다. 그런데—

3단계: 매 반복마다 **같은 캐시를 재사용**합니다. 왜? **이미지와 프롬프트는 그동안 안 바뀌니까요**. PaliGemma forward를 다시 돌릴 필요가 없는 거예요.

[말줄임] 이 캐싱 덕분에 **10회 디노이징을 73 ms에 끝낼 수 있어요**. 이게 RTX 4090 한 장에서 실시간 제어가 가능해진 결정적인 이유입니다.

수치로 잠깐 감 잡아봅시다. 73 ms당 한 번 디노이징 사이클이 돌아간다는 건 약 **13 Hz의 제어 주기**예요. 인간의 반응 속도보다 빠릅니다. 이게 실제 로봇이 자연스럽게 움직이는 데 충분한 속도입니다.

[23] π_0 의 한 step 전체 흐름

π_0 한 사이클 전체를 한번에 정리해 봅시다.

1단계: 카메라 세 장 + 텍스트 프롬프트 + 현재 robot state를 수집합니다.

2단계: PaliGemma forward pass를 한 번 돌려서, 모든 layer의 K, V를 캐시합니다.

3단계: Action Expert를 초기화합니다. 1개의 state token + 50개의 noise token이 input이예요.

4단계: Flow matching 루프를 **10회** 돌립니다. 각 회에서 self-attention과 (concatenated K/V를 통한) cross-attention을 동시에 수행하고, Δa 를 계산해서 궤적을 업데이트합니다.

5단계: 최종 14×50 궤적을 출력합니다.

6단계: 로봇이 이 궤적의 **처음 몇 step만 실행**합니다. 끝까지 가는 게 아니예요. 왜냐하면 환경이 변할 수 있으니까요.

7단계: 새 카메라 이미지를 받아서 **1단계로 돌아갑니다**. 이걸 약 13 Hz로 반복하는 거죠.

[말줄임] 이게 “model predictive control”의 일종으로 볼 수 있어요. **미래를 멀리 내다보고 계획하되, 일부만 실행하고 다시 계획하는** 패턴이죠. 고전 제어와 현대 학습 기반 정책의 흥미로운 결합입니다.

Part 7. 비판적 시각과 미래

[24] 데모를 신중히 읽기: 1995년 Ralph

지금까지 π_0 의 놀라운 능력을 봤어요. 그런데 우리가 연구자로서 한 발짝 떨어져서 봐야 할 게 있습니다.

1995년 이야기를 잠깐 할게요. Carnegie Mellon에서 Ralph라는 자율주행 시스템을 만들었어요. 미국을 **98.2% 자율로** 횡단했습니다. 1995년에요.

자, 이거 들으면 “와, 그럼 자율주행차가 1995년에 다 됐나?” 싶죠? 전혀 아닙니다. 30년이 지난 지금, 우리가 가진 자율주행차는 Ralph의 직계 후손이 아니에요. 센서 융합, HD map, learned planner, end-to-end transformer 같은 **전혀 다른 패러다임**의 조합이에요. 그리고 30년이 지나도 “완전히 풀린” 문제가 아닙니다.

[말줄임] π_0 도 마찬가지예요. **인상적인 데모와 “실제로 작동하는 시스템”은 다른 문제**입니다. 빨래 개기 데모가 인상적이라고 “집안일 다 해주는 로봇”이 코앞에 있다고 결론 내리면 안 돼요.

대학원생으로서, 그리고 미래의 연구자로서, 데모를 **비판적으로 읽는 능력**이 정말 중요합니다. “이 데모는 무엇을 보여주고, 무엇을 보여주지 않는가?” 이 질문을 항상 던져야 해요.

[25] 대안 패러다임: World Models

실제로 “VLA가 정말 올바른 길인가?”에 대한 의문을 강하게 제기하는 학자들이 있어요. 가장 대표적인 사람이 **Yann LeCun**이에요.

LeCun은 AI 분야의 대선구자고, Meta의 수석 AI 과학자였어요. 그런데 최근에 Meta를 떠나서 **world model 중심의 새 벤처**를 시작했습니다. 그의 방향은 **LLM을 백본으로 쓰지 않는 쪽**이에요. JEPA(Joint-Embedding Predictive Architecture) 계열이라고 합니다.

LeCun의 입장은 거침없어요. 인터뷰에서 “JEPA가 결국 VLA를 추월할 거라 보시나요?” 묻자, “물론이죠. VLA는 망했어요. 정말 잘 작동하지 않거든요”라고 답했습니다.

[말줄임] LeCun이 무슨 말을 하고 있는지 핵심을 정리해 봅시다.

VLA의 패러다임: 인터넷 텍스트와 이미지로 사전학습 → 행동 데이터로 fine-tune. “LLM이 곧 두뇌”라는 가정.

World model의 패러다임: 환경의 **예측 모델**을 latent space에서 학습. “다음 순간이 어떻게 될지” 시뮬레이션할 수 있는 능력 자체가 지능의 핵심이라는 가정. Planning은 학습된 world model 위에서 search나 optimization으로 합니다.

어느 쪽이 옳을까요? 솔직히 아무도 모릅니다. 다만 두 패러다임이 강조하는 **근본적으로 다른 가정**을 이해하는 게 중요해요. 다음 강의에서 world model을 자세히 다루겠습니다.

[26] 토론거리

오늘 마무리는 토론으로 합시다. 다섯 가지 질문 던져드릴게요. 정답이 있는 질문이 아니에요. 다음 주까지 자기 생각 정리해 와 주세요.

1. RT2의 일반화(Taylor Swift)는 진짜 “이해”인가, 아니면 통계적 보간일 뿐인가? 이걸 검증할 실험을 설계할 수 있을까?
2. Action Expert를 따로 두는 건 일종의 **귀납적 편향(inductive bias)**을 모델에 주입한 거예요. 이게 “순수 end-to-end” 철학과 충돌할까요, 아니면 보완할까요? 우리는 어디까지 architecture를 손봐야 할까요?
3. Flow matching이 mode collapse를 피한다고 했습니다. 그런데 **원치 않는 mode**—가령 안전하지 않은 동작—도 학습될 위험은 없을까요? 어떻게 막을 수 있을까요?
4. KV cache 공유는 “같은 아키텍처”에 의존합니다. 그러나 미래의 로봇 시스템이 **이종 모듈**을 결합해야 한다면(예를 들어 SLAM 모듈 + LLM + 제어 모듈), 더 일반적인 방법은 무엇이 있을까요?
5. (가장 우리 lab에 해당하는 질문) LeCun의 주장이 옳다면, classical SLAM과 state estimation은 미래 로봇 두뇌에서 어떤 자리를 차지할까요? 우리 APRL의 “Classical-to-Modern Bridge” 정체성을 어떻게 발전시켜야 할까요?

[말줄임] 다섯 번째 질문은 사실 우리 lab 정체성과 직결되는 부분이에요. ScaleMaster, LT-Mem, Mag4D-SLAM 모두 classical 기하학적 기초 위에서 modern learning을 결합하는 작업입니다. VLA 시대에서 이 결합의 의미를 어떻게 재정의할지가, 다음 5년의 우리 lab의 방향이 될 거라고 봐요.

[27] 핵심 정리

오늘 다룬 내용을 한 번에 정리합니다.

- 1. VLA의 정의:** Vision + Language + Action을 단일 모델에서 통합하는 패러다임. 인터넷 사전학습의 지식을 행동과 연결하는 게 핵심 가치.
- 2. π_0 의 세 가지 설계 원칙:** 첫째, 백본 LLM(PaliGemma) 위에 별도 Action Expert를 둔다. 둘째, Action Expert는 flow matching으로 궤적을 생성한다. 셋째, KV cache 공유로 두 모듈을 매끄럽게 결합한다.
- 3. 두 가지 깊은 통합:** 인터넷 지식 $\leftrightarrow$ 물리적 행동의 통합과, 이미지 생성 수학 $\leftrightarrow$ 로봇 제어 수학의 통합. 둘 다 “서로 다른 영역이 같은 프레임워크로 연결된다”는 통일성을 보여줍니다.
- 4. 한계와 대안:** World model 패러다임(LeCun, JEPa)은 “LLM이 곧 두뇌”라는 VLA의 가정을 거부합니다. 어느 쪽이 옳은지는 아직 열려 있습니다.
- 5. 연구자에게 주는 교훈:** 인상적인 데모와 실제로 작동하는 시스템 사이의 간극을 잊지 말 것. 1995년 Ralph가 우리에게 가르쳐 준 교훈입니다.

[말줄임] 마지막으로 한마디. 우리 lab은 **classical과 modern을 잇는 다리**라는 정체성을 갖고 있어요. VLA는 modern의 최전선이지만, 이게 잘 작동하려면 결국 **state estimation, geometric reasoning** 같은 classical 기초가 필요합니다. 정밀한 manipulation에는 결국 정확한 6DoF state가 필요한 것처럼요.

[28] 참고 문헌

오늘 강의 자료의 원천을 다시 명시합니다.

원본 영상: “Inside the World’s Smartest Robot Brain [VLA]”, **Welch Labs**.

URL: youtube.com/watch?v=2mrGMMmrVNE

영상이 매우 잘 만들어져 있으니 직접 보시는 걸 강력 추천합니다. 시각적 설명이 뛰어나요.

핵심 논문 시작점:

- Ahn et al. (2022), SayCan
- Brohan et al. (2022), RT-1
- Driess et al. (2023), PaLM-E
- Brohan et al. (2023), RT-2
- Black et al. (2024), π_0
- Lipman et al. (2023), Flow Matching
- LeCun (2022), JEPA position paper

[29] 다음 주 예고와 마무리

다음 주에는 **World Model**과 **JEPA 계열**을 깊이 다뤄볼 거예요.

오늘이 “LLM 기반 로봇 두뇌” 편이었다면, 다음 주는 “LLM에 의존하지 않는 로봇 두뇌” 편입니다. 두 패러다임을 직접 대비시켜 볼 거예요.

과제: 오늘 토론거리 다섯 개에 대한 자기 생각을 짧게라도 정리해서 다음 주에 가져와 주세요. 특히 다섯 번째—“classical SLAM의 미래 자리”—는 우리 lab의 방향과 직결되니까 신중히 생각해 보시기 바랍니다.

권장 예습: LeCun의 2022년 JEPA position paper(“A Path Towards Autonomous Machine Intelligence”). 길지만 도입부와 결론만 봐도 다음 주 흐름을 잡을 수 있어요.

[말줄임] 오늘 수고하셨습니다. 질문 있으면 받겠습니다.